

< 200 CONCRETE OPEN PROBLEMS IN MECHANISTIC INTERPRETABILITY >

200 COP in MI: Analysing Training Dynamics

by **Neel Nanda** 4th Jan 2023

This is the sixth post in a sequence called 200 Concrete Open Problems in Mechanistic Interpretability. [Start here](#)[°], then read in any order. If you want to learn the basics before you think about open problems, check out [my post on getting started](#). Look up jargon in [my Mechanistic Interpretability Explainer](#)

Disclaimer: *Mechanistic Interpretability is a small and young field, and I was involved with much of the research and resources linked here. Please take this sequence as a bunch of my personal takes, and try to seek out other researcher's opinions too!*

Motivating papers: [A Mechanistic Interpretability Analysis of Grokking](#)[°], [In-Context Learning and Induction Heads](#)

Background

Skip to motivation if you're familiar with the grokking modular addition and induction heads papers

Several mechanistic interpretability papers have helped find surprising and interesting things about the training dynamics of networks - understanding what is actually going on in a model during training. There's a lot of things I'm confused about here! How do models change over training? Why do they generalise at all (and *how* do they generalise)? How much of their path to the eventual solution is consistent across runs and directed vs a random walk? How

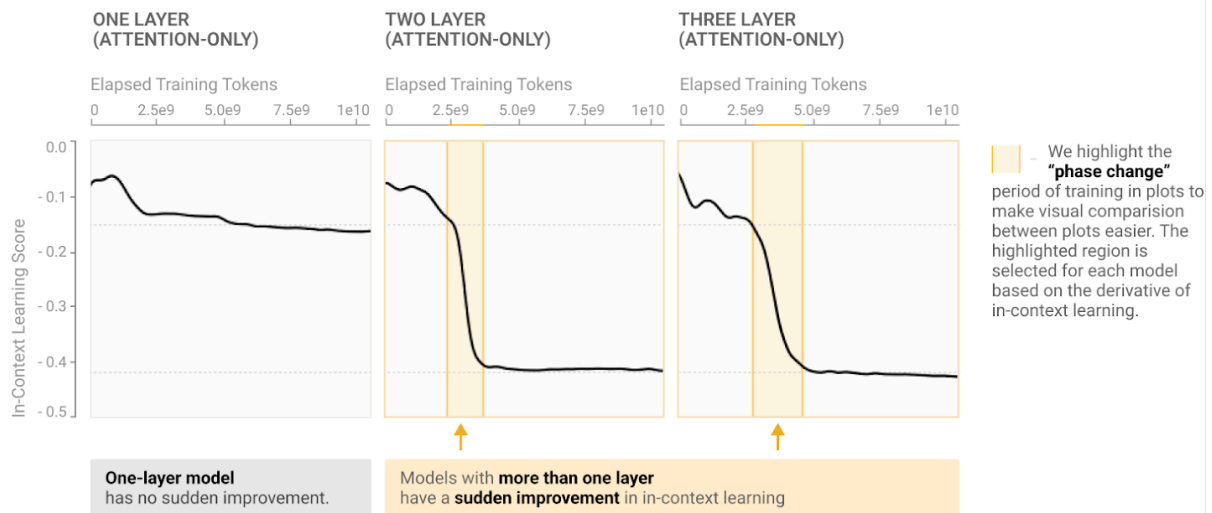
and when do the different circuits in the model develop, and how do these influence the development of subsequent circuits?

The questions here are of broad interest to anyone who wants to understand what the hell is going on inside neural networks, and there are angles of attack on this that are nothing to do with Mechanistic Interpretability. But I think that MI is a particularly promising approach. If you buy the fundamental claim that the building blocks of neural networks are interpretable circuits that work together to complete a task, then studying these building blocks seems like a grounded and principled approach. Neural networks are extremely complicated and confusing and it's very easy to mislead yourself. Having a good starting point of a network you *actually* understand makes it far easier to make real progress.

I am particularly interested in understanding **phase transitions** (aka emergent phenomena), where a specific capability suddenly emerges in models at a specific point in training. Two MI papers that get significant traction here are [A Mechanistic Interpretability Analysis of Grokking](#)^o, and [In-Context Learning and Induction Heads](#), and it's worth exploring what they did as models of mechanistic approaches to studying training dynamics:

In [In-Context Learning and Induction Heads](#), the focus was on finding phase changes in real language models. We found [induction heads/circuits](#) in two layer, attention-only language models, a circuit which performs the task of detecting whether the current text was repeated earlier in the prompt, and continuing it. These turned out to be a crucial circuit in these tiny models - the heads were principally responsible for the model's ability to do **in-context learning**, productively using tokens far back in the context (eg over 500 words back!) to predict the next token. Further, the induction heads all arose in a sudden phase transition, and were so important that this led to a visible bump in the loss curve!

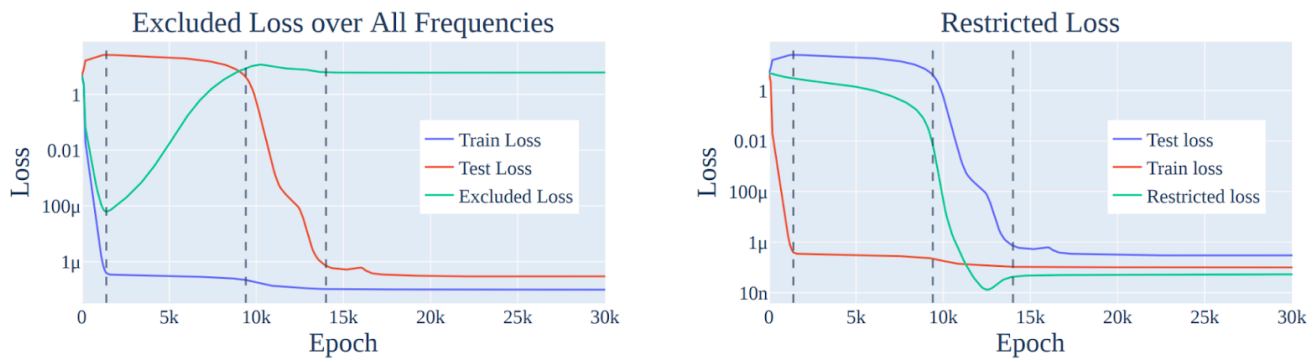
MODELS WITH MORE THAN ONE LAYER HAVE AN ABRUPT IMPROVEMENT IN IN-CONTEXT LEARNING



The focus of the paper was in **extrapolating** this mechanistic understanding in a simple setting to real models. We devised a range of progress measures (for both overall model behaviour and for studying specific heads), and analysed a wide range of models using these. And we found fairly compelling evidence that these results held up in general, up to 13B parameter models!

Grokking is a surprising phenomena where certain small models trained on algorithmic tasks initially memorise their training data, but when trained on the *same* data for a really long time, will suddenly generalise. In our grokking work, we focused on a 1 layer transformer that grokked modular addition. We did a deep dive into its internals and reverse engineered it to discover it had learned trig identity based algorithm.

We then distilled this understanding of the final checkpoint into a series of **progress measures**. We studied **excluded loss**, where we *just* removed the model's ability to use the generalising solution and looked at train performance, and **restricted loss** where we artificially remove the memorising solution and *only* allow the model to use the generalising solution and look at overall performance. These let us decompose the training dynamics into 3 discrete phases: **memorisation**, where the model just memorises the training data, **circuit formation**, where the model slowly learns a generalising circuit and *transitions* from the memorising solution to the generalising solution - preserving good train performance and poor test performance throughout, and **cleanup**, where it gets so good at generalising that it no longer needs the parameters spent on memorisation and can clean them up. The sudden emergence of "grokking" only occurs at cleanup, because even when the model is mostly capable of generalising, the "noise" of the memorising solution is sufficient to prevent good test loss.



Both works used a mechanistic understanding of a simpler setting to try to understand a confusing phenomena, but with somewhat different focuses. The approach modelled in grokking is one at the intersection of MI and the science of deep learning. We took a specific confusing phenomena in neural networks, **simplified** it to a simple yet still confusing example, **reverse-engineered** exactly what was going on, **extrapolated** it to the core principles (of progress measures) and analysed these during training. By deeply focusing on understanding a specific example of a specific phenomena, we (hopefully!) gained some general insights about questions around memorisation, generalisation, phase transitions, and competition between different circuits (though there's a *lot* of room for further work verifying this, and I'm sure some details are wrong!). (Check out the [Interpreting Algorithmic Problems post](#)° for another angle on this work). In contrast, the induction heads work has a bigger focus on showing the universality of the result across many models, studying real models in the wild, and being able to tie the zoomed in understanding of a specific model circuit to the important zoomed out phenomena of in-context learning.

Motivation

Zooming out, what are the implications of all this for future research? I think there's exciting work to be done, both on using toy models to find general principles of how neural networks learn and on understanding the training dynamics in real language models. A lot of the current low-hanging fruit is in fairly simple settings and building better fundamental understanding. But a good understanding of training dynamics and how it intersects with interpretability should eventually push on some important long-term questions:

- Helping us find ways to track and predict [emergent phenomena](#), such as by using a mechanistic understanding to find smoother progress measures that precede the emergence.
 - My personal prediction is that *most* circuits form in sudden phase transitions, but this all averages out into a fairly smooth loss curve, because each individual circuit is just a marginal contribution to model performance. From this perspective,

studying circuits is key to understanding emergence, since emergence is likely a consequence of some circuits suddenly developing.

- Helping understand the potential development of misaligned capabilities like deception or [a model learning incorrect proxy goals](#) that generalise poorly. Eg, if these do happen, should we expect them to develop suddenly, or continuously?
- More ambitiously, exploring how an understanding of interpretability might let us *change* training dynamics to change a model's trajectory and avoid these failure modes, such as by putting interpretability inspired metrics in the loss function.

Work on deep learning mysteries (in contrast to directly on real language models) doesn't directly push on the above, but I think that being less confused about deep learning will help a lot. And even if the above fail, I expect that exploring these questions will naturally open up a lot of other open questions and curiosities to pull on.

I'm particularly excited about this work, because I think that trying to decipher mysteries in deep learning makes mechanistic interpretability a healthier field. We're making some pretty bold claims that our approach will let us really reverse-engineer and fully understand systems. If this is true, we *should* be able to get much more traction on demystifying deep learning, because we'll have the right foundations to build on. One of the biggest ways the MI project can fail is by ceasing to be fully rigorous and losing sight of what's *actually* true for models. If we can learn true insights about deep learning or real language models, this serves both as a great proof of concept of the promise of MI, and grounds the field in doing work that is actually useful.

To caveat the above, understanding what on earth is going on inside networks as they train highly appeals to my scientific curiosity, but I don't think it's *necessary* for the goal of mechanistically understanding a fully trained system. My personal priority is to understand specific AI systems on the frontier, especially at human level and beyond. This is already a very ambitious goal, and I don't think that understanding training dynamics is obviously on the critical path for this (though it might be! Eg, if models are full of irrelevant "vestigial organs" learned early in training). And there's a risk that we find surface-level insights about models that allows us to make better models faster, while still being highly confused about how they *actually* work, and on net being even more behind in understanding frontier models. And models are already a confusingly high dimensional object without adding a time dimension in! But I could easily be wrong, and there's a lot of important questions here (just far more important questions than researchers!). And as noted above, I think this kind of work can be very healthy for the field. So I'd still encourage you to explore these questions if *you* feel excited about them!

Resources

- The [induction heads paper](#)
 - [A video walkthrough](#) I made with Charles Frye on it
 - The [Induction Circuits section](#) of my MI explainer
- My [grokking work](#)° and [the accompanying colab notebook](#)
 - If you want to work on a grokking related question, [email me](#) for a significantly more up to date draft
- My [TransformerLens library](#) supports many sequences of checkpointed models, see [model documentation here](#)
 - My toy models all have ~200 checkpoints taken on a roughly exponentially decaying schedule
 - The SoLU, GeLU and attn-only models of 1-4L were all trained with the same initialization and data shuffle (that is, all 1Ls are the same, all 2Ls are the same, etc). I have no idea whether this makes a difference, but at least some induction heads seem shared across them!
 - There are some larger SoLU models (up to GPT-2 Medium size), and an older scan trained on the Pile which both have checkpoints taken during training
 - The Stanford CRFM released 5 GPT-2 small and 5 GPT-2 medium sized models with 600 checkpoints and each with a different random seed (in TransformerLens as `stanford-gpt2-small-a` etc)
 - Eleuther's Pythia project has models from 19M parameters to 13B, with 143 linearly spaced checkpoints taken during training. Each model was trained on the exact same data shuffle.
 - They also contain a scan of models trained on a de-duplicated version of the Pile
- The [Training Dynamics section](#) of my MI explainer

Tips

- Studying training dynamics is generally easy to do with small models trained on algorithmic tasks - you can train them yourself fairly easily and take as many checkpoints as you want!
- If training a model takes more than 1-2 hours, I recommend taking a *lot* of checkpoints (unless you have major storage constraints).

- And make sure to set a random seed at the start! You *really* want to be able to reproduce a training run, in case you want to take more checkpoints, add some new metrics, etc.
- I recommend taking more checkpoints earlier in training (eg on an exponentially decaying schedule), there tends to be more interesting stuff in the first few steps/epochs.
- It's easy to jump too fast to studying training. I recommend first ensuring you really understand the *final* model, and use this understanding to ground your explorations during training.
 - Though it can also be useful to go back and forth - briefly exploring the model during training can help you figure out what's most relevant to study
- If you're training your own models, you should save checkpointed runs with several different random seeds. This is an easy way to sanity check your results!
- Studying training dynamics can be significantly harder than reverse engineering a single model, because time is an additional dimension to grapple with (and networks have enough dimensions as it is!).
 - One consequence of this is that, even more so than normal research, you want to be careful about prioritisation. There's often a lot of weird shit during training, and you want to focus on what's most relevant to the question you're trying to answer.
- Phase transition is a somewhat fuzzy concept, and different metrics can exaggerate/underrate them. As a rough rule of thumb, metrics like cross-entropy loss are continuous and smoother, metrics like accuracy are discrete and sharper (so look more phase transitiony). And things look sharper with a log scale on the x axis. (A fun exercise is comparing how different phase transition focused papers present their data!)
 - It's not clear to me what the *right* metric to use here is - if something only looks like a phase transition under accuracy, then that's less compelling, but still clearly tells you *something*. I recommend using both kinds, and plotting graphs with linear or log scale axes.

Problems

[This spreadsheet](#) lists each problem in the sequence. You can write down your contact details if you're working on any of them and want collaborators, see any existing work or reach out to other people on there! (thanks to Jay Bailey for making it)

Note: *Many of these questions are outside my area of expertise, and overlap with broader science of deep learning efforts. I expect there's a lot of relevant work I'm not familiar with, and I would not be massively surprised if some of these problems have already been solved!*

- Algorithmic tasks
 - Understanding grokking
 - **B* 5.1** - Understanding why [5 digit addition has a phase change](#)° per digit (so 6 total?!)
 - C 5.2 - Why in the order it does?
 - **B* 5.3** - Look at the PCA of logits on the full dataset, or look at the PCA of a stack of flattened weights. If you plot a scatter plot of the first 2 components, the different phases of training are clearly visible. What's up with this?
 - Can you interpret the different components? (Very roughly, I think one of them is memorising circuit - generalising circuit)
 - Can you use these to predict *when* the model will grok? I've had some success doing this with the components calculated using *all* checkpoints, but haven't figured how to do it without future information.
 - **C* 5.4** - Can we predict *when* grokking will happen? A metric that doesn't use any future information would be awesome
 - **C* 5.5** - Understanding why the model chooses specific frequencies (and [why it switches mid-training sometimes!](#))
 - B-C 5.6 - What happens if we include in the loss one of the progress measures in my post/paper - can we accelerate or stop grokking?
 - **B* 5.7** - [Adam Jermyn](#)° provides an analytical argument and some toy models for why phase transitions should be an inherent part of (some of) how models learn. Can you find evidence of this in more complex models?
 - I would start by training a 2L attn only model on repeated random tokens to form induction heads. This has a phase transition, can you figure out why?
 - B 5.8 - Build on and refine his arguments and toy models, eg thinking about the ways in which they deviate from a real transformer, and building toy models that are more faithful
 - Lottery Tickets
 - **B-C* 5.9** - Eg for a toy model trained to form induction heads. Is there a lottery-ticket style thing going on? Can you disrupt induction head

formation by messing with the initialization? (eg train 4 models, find the least induction-y heads in each, and initialize a model with those heads)

- What happens if you initialize the model with the fully trained prev token or induction head?
 - **C* 5.10** - All of my toy models with n=1 to 4 layers (attn-only, gelu and solu) were trained with the same data shuffle and weight initialization. [Looking at the induction heads](#) in the models of the, many are not shared, but head L2H3 in the 3L ones and head L1H6 in the 2L ones are induction heads always. What's up with that?
 - B 5.11 - If we knock out the parameters that form important circuits at the end of training on some toy task (eg modular addition) at the *start* of training, how much does that delay/stop generalisation?
- Analysing how the pair of heads compose in [an induction circuit](#) over time (previous token head and induction head)
 - **B* 5.12** - Can you find progress measures which predict these?
 - Can this be detected with [composition scores](#)? Can you refine this into something that will?
 - **B* 5.13** - At initialization, can we predict *which* heads will learn to compose first? Is it at all related to how much they compose at initialization?
 - If we copy this pair of heads into another randomly initialized model, does this pair still compose first?
 - B 5.14 - Does the composition develop as [a phase transition](#)?
 - What do the compositions scores look like over training?
- Understanding fine-tuning
 - **C* 5.15** - Build a toy model of fine-tuning (where the model is pre-trained on task 1, and then fine-tuned on task 2). What is going on internally? Can you find any interesting motifs?
 - Possible variations - totally switching tasks, varying just one aspect of the task, or training 50% on the new task and 50% on the old task.
 - See the Superposition post for discussion on how to approach fine-tuning projects
 - What happens within a model when you fine tune it? TransformerLens contains versions of the 1L SoLU and 4L SoLU toy models fine-tuned on 4.8B tokens of wikipedia ([documented here](#), load with `solu-1l-wiki` or `solu-4l-wiki`, 163 checkpoints taken during fine-tuning are also available)

- Hypothesis: Fine-tuning is mostly just rewiring and upweighting vs downweighting circuits that already exist, rather than building new circuits.
- **A* 5.16** - Explore how model performance change on the original training distribution. Are specific capabilities harmed or is worse across the board?
- **B* 5.17** - How is the model different on the fine-tuned text? I'd look for examples of fine-tuning text where the model does *much* better post fine-tuning, and start there, but also look at some more normal text.
 - Start with direct logit attribution and compare model components - do all matter a bit more, or do some matter way more and the rest are the same?
 - **B*** Try activation patching between the old and new model on the same text and see how hard it is to recover performance - do some components change a lot, or do all change a bit?
- **B* 5.18** - Look at max activating text for various neurons in the original models. How has it changed post fine-tuning? Are some neurons overwritten, are most basically the same, or what?
 - I'm particularly curious about whether the model learns new features by shoving them in in superposition, while keeping all of the old features, vs significantly rewriting things.
- **A-C* 5.19** - Explore further and see what you can learn about what is going on mechanistically with fine-tuning (I predict that you can easily learn *something*, but that there's a lot of room to dig deep and find insights)
- **B-C* 5.20** - Can you find any phase transitions in the checkpoints taken during fine-tuning?
- Understanding training dynamics in language models
 - Phase transitions
 - **A-B* 5.21** - Can you replicate the induction head phase transition results in the various checkpointed models in TransformerLens? (I'd write code that works for `attn-only-21`, and then if you just change the pretrained model name, the code should work for all of the models!)
 - **B* 5.22** - Look at the neurons in my SoLU models during training. Do they tend to form as a phase transition?
 - The easiest place to start would be taking the max activating dataset examples from [neuroscope](#), running those through the model, and

looking at how the activation changes (though you'll need to significantly refine this to be confident there's a phase transition!)

- **C* 5.23** - Use the [per-token loss analysis](#) technique from the induction heads paper to look for more phase changes - when plotting the first two dimensions the induction head phase change shows up as a clear turn. Can you figure out a capability or circuit that these correspond to?
 - I'd start with other toy models (eg `solu-2l`, `solu-1l` or `attn-only-3l`)
 - You can also try out [other dimensionality reduction techniques](#)
 - I would look for the first 3 components, or only study checkpoints after the induction bump, so you don't *just* see the induction bump!
- Other ideas for finding phase transitions in real language models, eg using [the Stanford CRFM models](#) or my toy models
 - **A* 5.24** - Look at attention heads on various texts and see if any have recognisable attention patterns (eg start of word, adjective describing current word, syntactic features of code like indents or variable definitions, most recent open bracket, etc), then analyse these over training.
 - A 5.25 - The [Indirect Object Identification task](#) (note - I've seen suggestive results that this doesn't have a phase change)
 - **B* 5.26** - Try digging into the specific heads that act on the problem. Use direct logit attribution for the name movers
 - **B* 5.27** - Or study the attention patterns of each category of heads
 - **B-C* 5.28** - Simple IOI-style algorithmic tasks - eg few shot learning, addition, sorting words into alphabetical order, completing a rhyming couplet, matching open and close brackets
 - B 5.29 - Soft induction heads, eg [translation](#), if any can do it
 - C 5.30 - Performance on benchmarks or specific questions from a benchmark
 - **D* 5.31** - Hypothesis: The reason scaling laws happen is that models experience a *ton* of [tiny phase changes](#)°, which average out to a smooth curve because of the law of large numbers. Can you find evidence for or against that? Are phase changes everywhere?
- Studying path dependence - how much do models trained in similar ways generally converge on the same behaviour (**path independence**), vs subtle

differences (or just different seeds!) leading to significant variation (**path dependence**)?

- **A-B* 5.32** - How much do the Stanford CRFM models have similar outputs on any given text, vs varying a lot?
 - Hypothesis - they'll all figure out circuits at approx the same rate, but with some noise. So they'll agree on the easy stuff but not the hard stuff. But the medium size ones will be able to do all the stuff the small ones can do.
 - B 5.33 - Try this out for algorithmic-flavoured tasks, like acronyms, IOI, see the [Circuits In the Wild post](#)° for more
- **A-B* 5.34** - Look for the IOI capability in other models of approx the same size? (OPT small, GPT-Neo small, Stanford CRFM models, `solu-8l`, `solu-10l`). Is it implemented in the same way?
 - I'm particularly interested in the Stanford models, because the *only* difference is the random seed.
- **B* 5.35** - When model scale varies (eg GPT-2 Small vs GPT-2 Medium) is there anything the smaller one can do that the larger one *can't* do? I'd start by looking at the difference in per token log prob.
 - My [toy models](#) or [SoLU models](#) have fairly small gaps between model sizes, and may be interesting to study here.
- **B* 5.36** - Try applying the [Git Re-Basin](#) techniques to a 2L MLP trained for modular addition. a) Does this work and b) If you use [my grokking work](#)° to analyse the circuits involved, how does the re-basin technique map onto the circuits involved?
 - Git Re-Basin is a technique for linearly interpolating between the weights of two models trained on the same task
 - (Note: My prediction is that this just won't work for modular addition, because there are different basins in model space for the different frequencies)
 - See [Stanislav Fort](#)'s somewhat negative replication for a starting point and criticism.
 - C-D 5.37 - Can you find some problem where you understand the circuits and where Git Re-Basin *does* work?

Previous:**200 COP in MI: Exploring
Polysemanticity and Superposition**

1 comments 34 karma

Next:**200 COP in MI: Techniques, Tooling and
Automation**

No comments 13 karma

[Log in to save where you left off](#)

Mentioned in

- 41 200 Concrete Open Problems in Mechanistic Interpretability: Introduction
- 10 200 COP in MI: Interpreting Reinforcement Learning

[Moderation Log](#)

More from Neel Nanda

19	Building and evaluating model diffing agents	bilalchughtai, Josh Engels, Neel Nanda	6h	0
32	Models May Behave Worse When Eval Aware...	Senthooran Rajamanoharan, Neel ...	2d	0
65	models have some pretty funny attractor sta...	aryaj, Senthooran Rajamanoharan, ...	3mo	0

View more

Curated and popular this week

39	Sympathy for both sides of the egregious misalignment debate	Steven Byrnes	7h	5
26	A Mike's-Eye View of ARC's Research	Mikewins	3d	0
19	Building and evaluating model diffing agents	bilalchughtai, Josh Engels, Neel Nanda	6h	0